Práctica 1: Explorando Linux embebido en la Raspberry Pi

Belmont Muñoz Samuel
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 317318289

Domínguez Palacios Jesús Alejandro
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 421066205

Dorantes Mejía Emmanuel
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 313004766

García Olvera Emiliano
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 314256009

Hernández Alonso Antonio Adonahí
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 316339825

Jiménez Reyes Alan Yael
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 320026726

Práctica No.: 1 | **Grupo:** 6 | **Fecha:** 07/09/2026

***Resumen*—Este reporte documenta la configuración del entorno de desarrollo en un sistema Linux embebido sobre Raspberry Pi 4 Model B Rev 1.5. Se describe la actualización del sistema operativo, la instalación de herramientas de compilación (toolchain), y la creación de programas en C y scripts de Bash para diagnóstico del sistema. El documento aborda la gestión de paquetes con APT, la inspección de configuraciones de bajo nivel a través de `/proc` y `/sys`, y el uso de Git para control de versiones. Se incluye el desarrollo de un programa que extrae información real del hardware y un script automatizado de monitoreo del sistema.**

***Index Terms*—Linux embebido, Raspberry Pi, toolchain, gcc, Git, scripts Bash, programación en C**

I. Objetivo

Los objetivos de esta práctica son:

- Actualizar el sistema y gestionar paquetes con APT.
- Instalar y verificar gcc, make y git.
- Inspeccionar la configuración de arranque y el árbol de dispositivos.
- Escribir un programa en C que reporte arquitectura, CPU y memoria reales del sistema.
- Combinar comandos de diagnóstico en un script de Bash.
- Registrar el primer commit de Git en un repositorio que se usará el resto del semestre.

Al finalizar la práctica se debe tener: el programa `reporte_sistema` compilando y mostrando datos reales, el script `sysinfo` con permiso de ejecución y su archivo de registro generado, y el historial de Git mostrando al menos un commit.

II. Introducción

El desarrollo de aplicaciones para sistemas embebidos requiere un entorno de trabajo adecuadamente configurado con herramientas de compilación, depuración y control de versiones. En sistemas Linux embebidos como Raspberry Pi OS, estas herramientas conforman lo que se conoce como *toolchain* de desarrollo [1].

Un toolchain típico incluye el compilador GCC (*GNU Compiler Collection*) para transformar código fuente en ejecutables nativos, Make para automatizar el proceso de compilación mediante archivos Makefile, y Git para el control de versiones distribuido del código fuente. Estas herramientas son fundamentales en el flujo de trabajo de desarrollo embebido profesional.

La gestión de paquetes en Raspberry Pi OS se realiza mediante APT (*Advanced Package Tool*), el sistema de administración de software heredado de Debian. APT permite instalar, actualizar y eliminar software de repositorios oficiales y comunitarios, manteniendo el sistema actualizado con parches de seguridad y nuevas funcionalidades.

En cuanto a la inspección del sistema a bajo nivel, Linux expone información de hardware y configuración del kernel a través de los sistemas de archivos virtuales `/proc` y `/sys` [2]. Archivos como `/proc/cpuinfo`, `/proc/meminfo` y `/boot/firmware/config.txt` proporcionan datos esenciales sobre el procesador, memoria y configuración de arranque. El árbol de dispositivos (*Device Tree*) es un mecanismo utilizado en arquitecturas ARM para describir el hardware disponible al kernel durante el arranque [4].

La programación en C sigue siendo el lenguaje dominante en sistemas embebidos debido a su cercanía al hardware, eficiencia y portabilidad. La capacidad de leer archivos del sistema virtual permite que programas en C obtengan información dinámica del hardware sin depender de bibliotecas externas pesadas.

Por otro lado, los scripts de Bash automatizan tareas administrativas y de diagnóstico. Un script bien diseñado puede ejecutar comandos del sistema, procesar su salida y generar reportes, todo de forma reproducible y documentada [3].

III. Desarrollo

III-A. Material y Equipo

Para la realización de esta práctica se utilizaron los siguientes elementos:

- Raspberry Pi 4 Model B Rev 1.5 con Raspberry Pi OS (base Debian *trixie*), kernel `6.18.39+rpt-rpi-v8`, con nombre de host `konan`
- Fuente de alimentación USB-C y tarjeta microSD de 64 GB con el sistema instalado en la Práctica 0
- Computadora host con cliente SSH
- Conexión a Internet para la descarga de paquetes

El trabajo se organizó rotando los roles de *piloto* (escribe y ejecuta los comandos) y *copiloto* (lee el paso, verifica la salida y registra las diferencias) en cada una de las siete partes de la práctica.

III-B. Procedimiento

III-B1. Parte A: Actualización del sistema y toolchain: Se estableció la conexión remota por SSH desde la computadora host hacia la Raspberry Pi, usando el nombre de host configurado en la Práctica 0. Ya dentro de la sesión, se actualizó el índice de paquetes y posteriormente el sistema completo.

```
sudo apt update
sudo apt upgrade -y
```

Listing 1. Actualización del sistema con APT.

El primer comando descarga las listas de paquetes desde los repositorios configurados; el segundo instala las versiones nuevas de los paquetes ya presentes en el sistema. La bandera `-y` evita la confirmación interactiva.

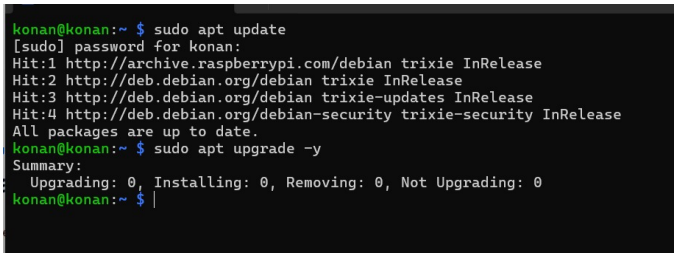


Figura 1. Actualización del índice de paquetes y del sistema mediante APT.

Enseguida se instaló el toolchain de desarrollo en C junto con el sistema de control de versiones. El metapaquete `build-essential` agrupa el compilador GCC, las bibliotecas estándar de desarrollo y la utilidad `make`.

```
sudo apt install -y build-essential git

gcc --version
make --version
git --version
```

Listing 2. Instalación y verificación del toolchain.

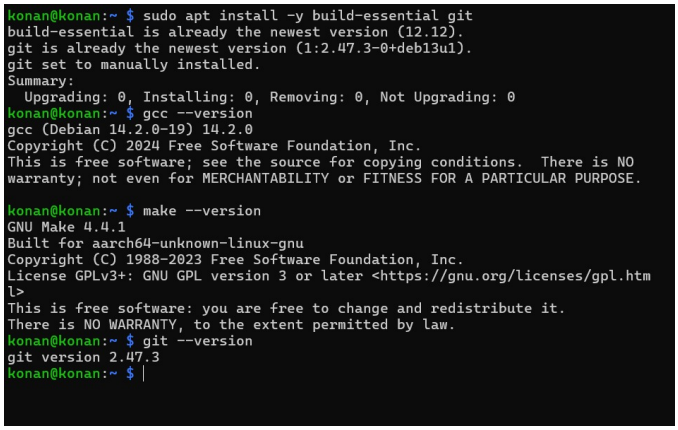


Figura 2. Instalación del toolchain y verificación de las versiones de gcc, make y git.

III-B2. Parte B: Configuración de arranque y árbol de dispositivos: Se inspeccionó el archivo de configuración de arranque, que en las versiones de Raspberry Pi OS basadas en Bookworm y posteriores reside en `/boot/firmware/config.txt` (en versiones anteriores la ruta era `/boot/config.txt`). Este archivo lo lee el firmware antes de que el kernel tome el control, y en él se declaran parámetros como los *overlays* del árbol de dispositivos, la habilitación de interfaces (I^2C, SPI, UART) y la configuración de video.

```
cat /boot/firmware/config.txt
ls /boot/firmware/
```

Listing 3. Inspección de la partición de arranque.

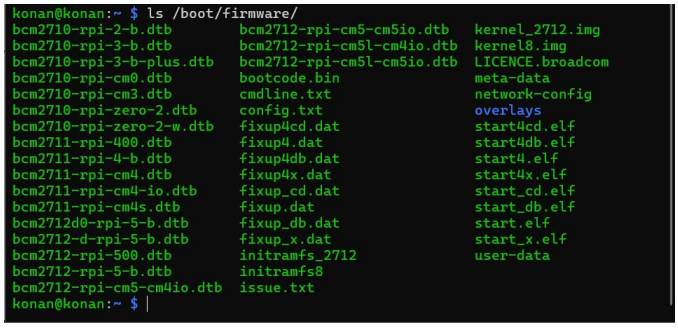


Figura 3. Contenido de la partición de arranque: firmware del GPU, imágenes del kernel y archivos `.dtb` de árbol de dispositivos.

Posteriormente se exploró la representación en vivo del árbol de dispositivos, es decir, la versión ya cargada e interpretada por el kernel y expuesta como sistema de archivos virtual.

```
ls /proc/device-tree/
```

Listing 4. Exploración del device tree activo.

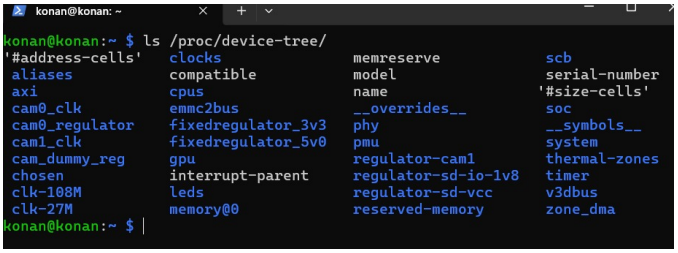


Figura 4. Nodos del árbol de dispositivos detectados por el kernel.

III-B3. Parte C: Procesos y servicios: Para observar los procesos activos se emplearon dos herramientas complementarias: `ps aux`, que entrega una fotografía estática de todos los procesos del sistema, y `top`, que muestra la misma información actualizándose en tiempo real y ordenada por consumo de CPU.

```
ps aux
top
```

Listing 5. Inspección de procesos activos.

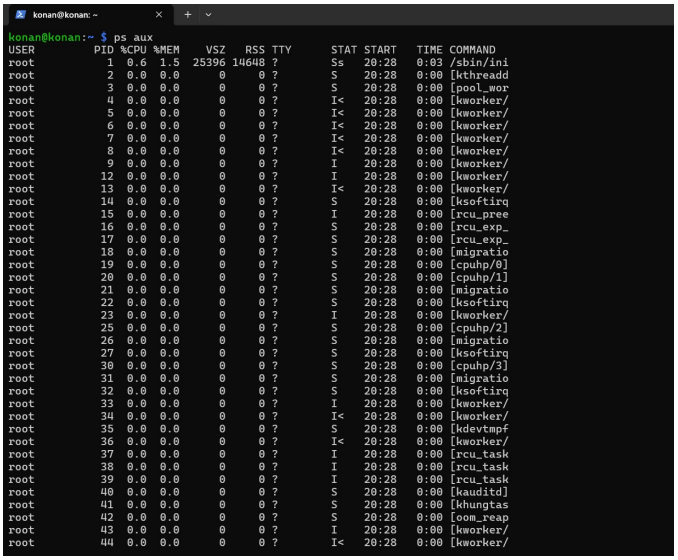


Figura 5. Listado de procesos activos con `ps aux`.

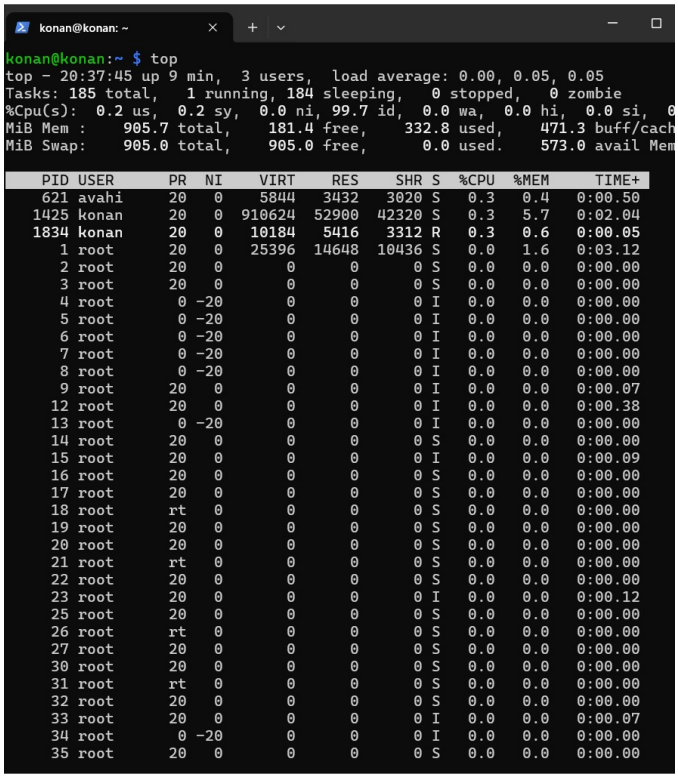


Figura 6. Monitor de procesos en tiempo real `top`, con el resumen de tareas, uso de CPU y memoria.

Finalmente se revisó el estado del gestor de servicios `systemd`, responsable de arrancar y supervisar los servicios del sistema.

```
systemctl status
systemctl list-units --type=service | head -n 15
```

Listing 6. Revisión de servicios del sistema.

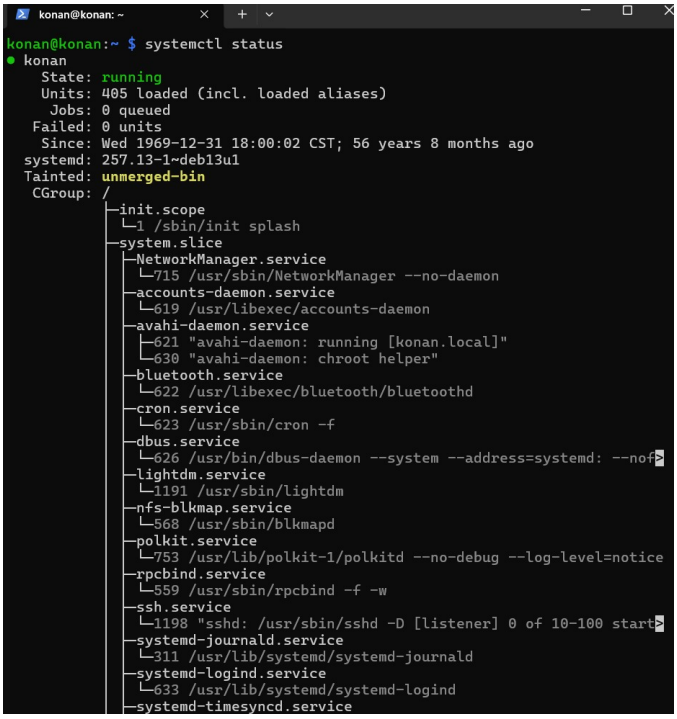


Figura 7. Estado general del sistema y árbol de grupos de control reportado por `systemd`.

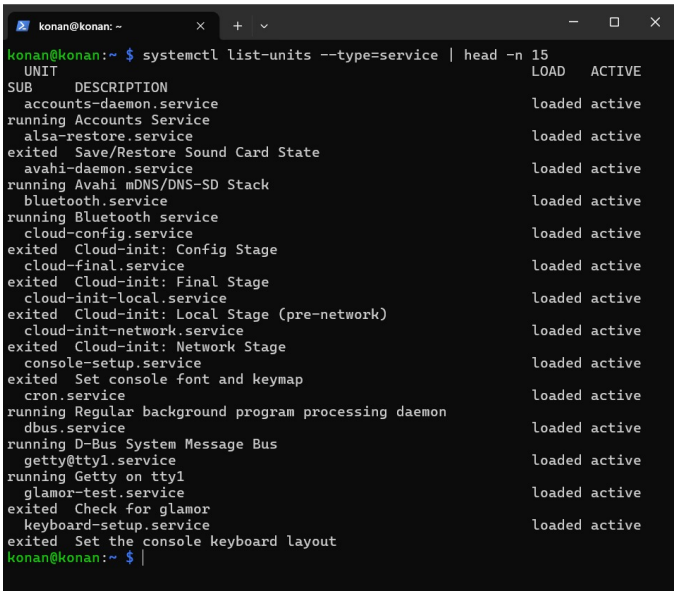


Figura 8. Primeras quince unidades de tipo servicio con su estado de carga y actividad.

III-B4. Parte D: Programa en C de reporte del sistema: Se escribió el programa `reporte_sistema.c`, que obtiene información del sistema por dos vías distintas: la llamada `uname()` de la biblioteca estándar, que entrega arquitectura y versión del kernel a través de la estructura `struct utsname`, y la lectura directa de los archivos virtuales `/proc/cpuinfo` y `/proc/meminfo`.

Para evitar duplicar código en la lectura de ambos archivos se implementó la función auxiliar `mostrar_linea()`, que recorre un archivo abierto línea por línea con `fgets()` y compara el inicio de cada línea contra una etiqueta usando `strncmp()`. Al encontrar la primera coincidencia la imprime y termina la búsqueda. Para el reto opcional se agregó la función `valor_kb()`, que en lugar de imprimir la línea completa extrae con `sscanf()` el valor numérico asociado a la etiqueta y lo devuelve como `long`, o `-1` si no la encuentra.

```c
#include <stdio.h>
#include <string.h>
#include <sys/utsname.h>

/* Busca la primera linea que empiece con 'etiqueta' y la
     imprime. */
void mostrar_linea(FILE *f, const char *etiqueta) {
  char linea[256];
  while (fgets(linea, sizeof(linea), f)) {
    if (strncmp(linea, etiqueta, strlen(etiqueta)) == 0) {
      printf("%s", linea);
      break;
    }
  }
}

/* Reto opcional: devuelve el valor numerico (en kB) de
     una etiqueta
   de /proc/meminfo, o -1 si no la encuentra. */
long valor_kb(FILE *f, const char *etiqueta) {
  char linea[256];
  long valor;
  while (fgets(linea, sizeof(linea), f)) {
    if (strncmp(linea, etiqueta, strlen(etiqueta)) == 0) {
      if (sscanf(linea + strlen(etiqueta), " : %ld",
           &valor) == 1)
        return valor;
    }
  }
  return -1;
}

int main(void) {
  struct utsname info;
  uname(&info);

  printf("=== Reporte del sistema ===\n");
  printf("Arquitectura: %s\n", info.machine);
  printf("Kernel:       %s\n", info.release);

  FILE *cpu = fopen("/proc/cpuinfo", "r");
  if (cpu) {
    printf("\n-- CPU --\n");
    mostrar_linea(cpu, "Model");
    mostrar_linea(cpu, "Hardware");
    fclose(cpu);
  }

  FILE *mem = fopen("/proc/meminfo", "r");
  if (mem) {
    printf("\n-- Memoria --\n");
    mostrar_linea(mem, "MemTotal");
    mostrar_linea(mem, "MemFree");
    rewind(mem);
    long libre = valor_kb(mem, "MemFree");
    if (libre >= 0)
      printf("Memoria libre: %.1f MB\n", libre / 1024.0);
    fclose(mem);
  }

  return 0;
}
```

Listing 7. Código fuente de `reporte_sistema.c`.

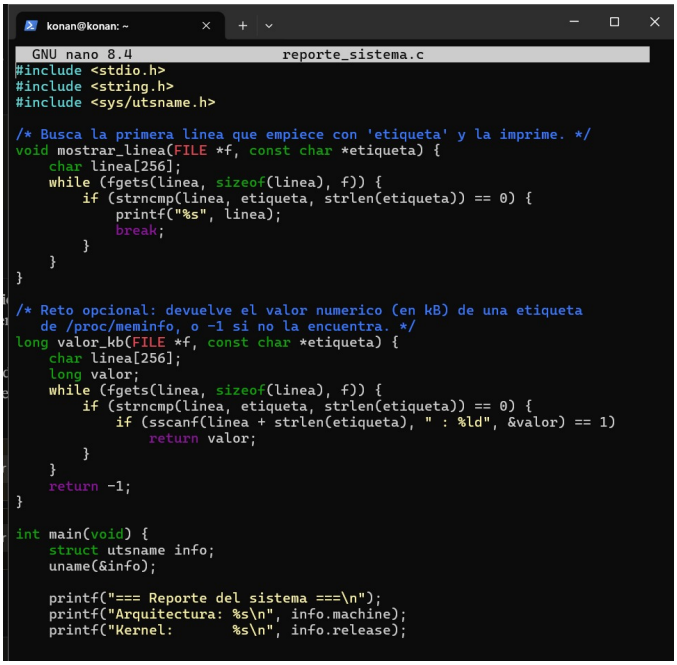


Figura 9. Edición del programa `reporte_sistema.c` en el editor `nano`.

La compilación y ejecución se realizaron con:

```
gcc reporte_sistema.c -o reporte_sistema
./reporte_sistema
```

Listing 8. Compilación y ejecución del programa.

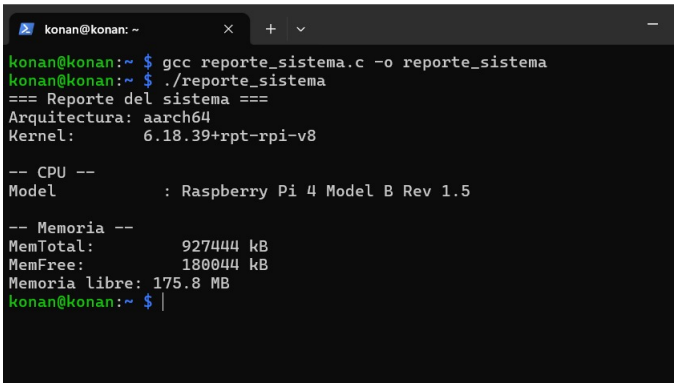


Figura 10. Salida del programa `reporte_sistema` con datos reales del hardware.

III-B5. Parte E: GPIO por software: Sin conectar hardware externo, se verificó que el kernel siguiera exponiendo la interfaz de propósito general de entrada/salida a través del sistema de archivos `sysfs`.

```
ls /sys/class/gpio/
```

Listing 9. Verificación de la interfaz GPIO.

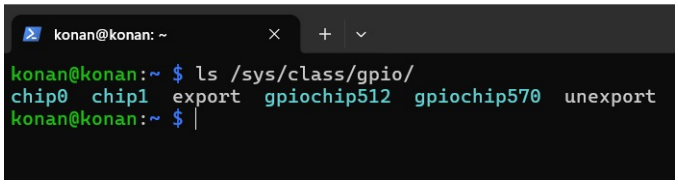


Figura 11. Interfaz GPIO expuesta por el kernel en `sysfs`.

III-B6. Parte F: Script de diagnóstico en Bash: Se creó el script `sysinfo.sh`, que agrupa varios de los comandos de diagnóstico utilizados durante la práctica y guarda la salida completa en un archivo de registro con marca de tiempo en el nombre. El bloque de comandos se agrupa entre llaves y se canaliza hacia `tee`, lo que permite mostrar la salida en pantalla y escribirla en el archivo simultáneamente.

```
#!/bin/bash
LOG="sysinfo_$(date +%Y%m%d_%H%M%S).log"
{
  echo "=== Reporte generado el $(date) ==="
  echo "--- Sistema ---"; uname -a
  echo "--- CPU ---";     lscpu | head -n 10
  echo "--- Memoria ---"; free -h
  echo "--- Disco ---";   df -h /
  echo "--- Procesos ---"; ps aux --sort=-%cpu | head -n 6
} | tee "$LOG"
echo "Reporte guardado en: $LOG"
```

Listing 10. Código fuente de `sysinfo.sh`.

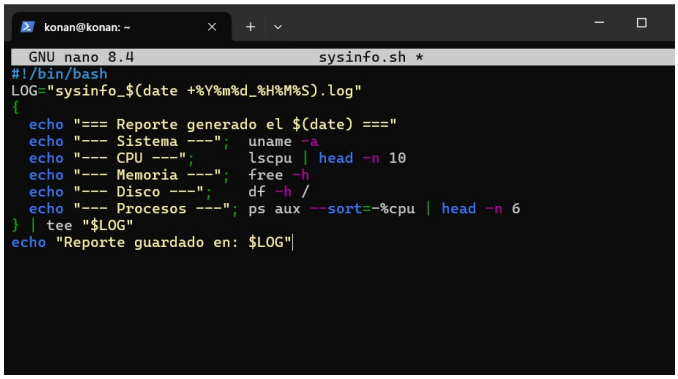


Figura 12. Edición del script `sysinfo.sh` en `nano`.

Como el archivo se crea sin permiso de ejecución, fue necesario otorgarlo antes de correrlo:

```
chmod +x sysinfo.sh
./sysinfo.sh
cat sysinfo_*.log
```

Listing 11. Permisos y ejecución del script.

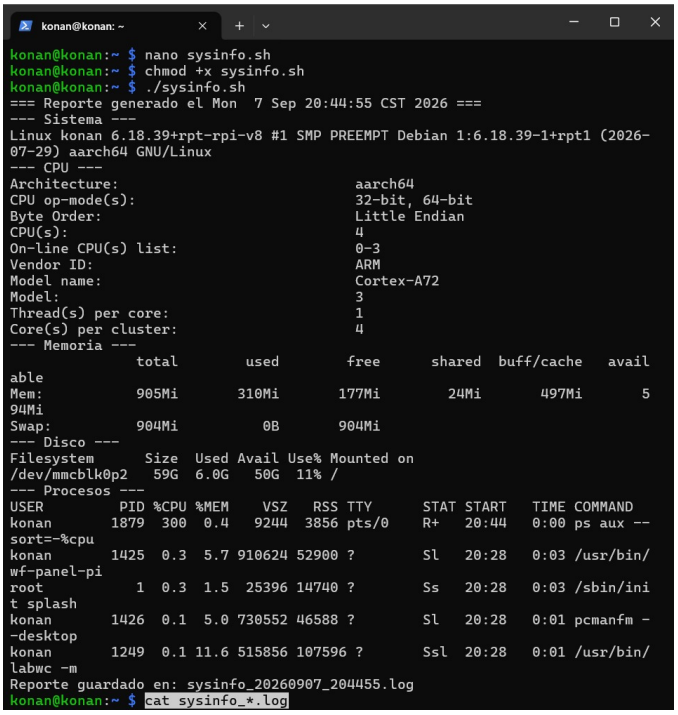


Figura 13. Ejecución del script tras otorgar el permiso de ejecución.

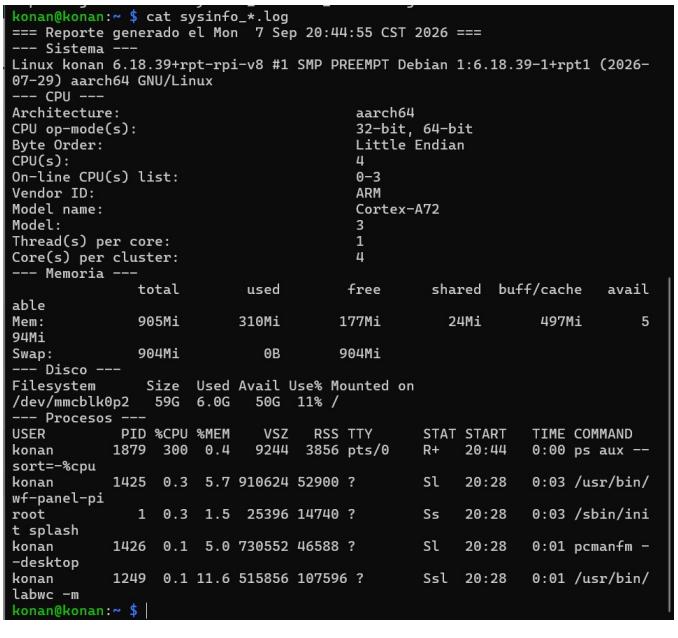


Figura 14. Contenido del archivo de registro `sysinfo_20260907_204455.log` generado por el script.

III-B7. Parte G: Primer commit con Git: Se configuró la identidad del usuario a nivel global en la Raspberry Pi, requisito indispensable para que Git pueda firmar los commits. El repositorio se creó dentro del directorio `~/fse`, que será el mismo que se utilice en las prácticas siguientes.

```
git config --global user.name "J Alejandro Dominguez"
git config --global user.email "adp3993@gmail.com"

git init
git add reporte_sistema.c sysinfo.sh
git commit -m "Practica 1"
git log
```

Listing 12. Configuración de identidad e inicialización del repositorio.

Se agregaron al área de preparación únicamente los archivos fuente producidos durante la sesión, dejando fuera el binario compilado y los archivos de registro, ya que son artefactos regenerables. Dado que el primer commit se realizó antes de terminar de configurar el correo electrónico, se corrigió la autoría y se amplió el mensaje con los nombres del equipo mediante:

```
git commit --amend --reset-author -m "Practica 1

Belmont Munoz Samuel
Dominguez Palacios Jesus Alejandro
Dorantes Mejia Emmanuel
Garcia Olvera Emiliano
Hernandez Alonso Antonio Adonahi
Jimenez Reyes Alan Yael"
```

Listing 13. Corrección de autoría del commit.

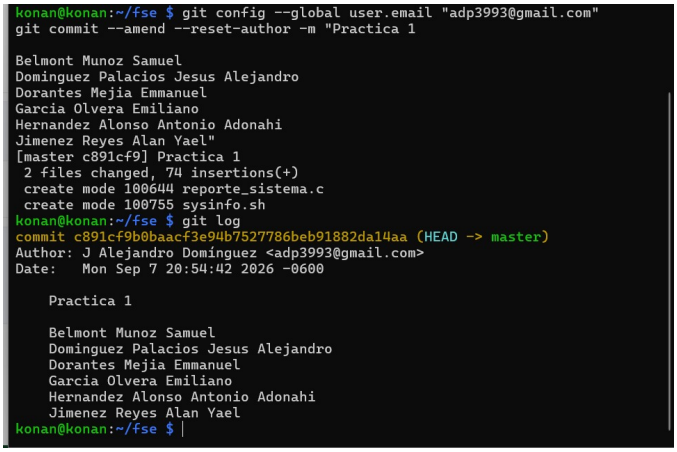


Figura 15. Corrección de autoría e historial del repositorio mostrando el commit `c891cf9`.

IV. Resultados y Análisis

IV-A. Actualización y toolchain

El comando `apt update` contactó cuatro repositorios: el de Raspberry Pi (`archive.raspberrypi.com/debian trixie`) y tres de Debian (`trixie`, `trixie-updates` y `trixie-security`), reportando que todos los paquetes ya estaban al día. En consecuencia, `apt upgrade -y` resumió cero paquetes actualizados, instalados o eliminados (Fig. 1). El sistema ya se encontraba actualizado desde la instalación realizada en la Práctica 0.

De forma análoga, `build-essential` (versión 12.12) y `git` (1:2.47.3-0+deb13u1) ya estaban instalados, por lo que APT únicamente los marcó como instalados manualmente. Las versiones verificadas se resumen en la Tabla I.

Cuadro I
VERSIONES DEL TOOLCHAIN INSTALADO.

Herramienta	Versión
GCC	14.2.0 (Debian 14.2.0-19)
GNU Make	4.4.1
Git	2.47.3
build-essential	12.12

Un detalle relevante es la línea *Built for aarch64-unknown-linux-gnu* que reporta Make: confirma que las herramientas fueron compiladas para la misma arquitectura sobre la que se ejecutan. Se trata entonces de compilación nativa, donde el mismo equipo que ejecuta el programa lo genera. Esto contrasta con el esquema de compilación cruzada usado en microcontroladores sin sistema operativo, donde el binario se produce en una máquina host de arquitectura distinta y luego se transfiere al dispositivo.

IV-B. Arranque y árbol de dispositivos

El listado de `/boot/firmware/` (Fig. 3) reveló que la partición de arranque no contiene únicamente los archivos de este modelo, sino los de toda la familia. Se identificaron tres grupos de archivos `.dtb`: los `bcm2710-*` (Raspberry Pi 3 y Zero 2 W), los `bcm2711-*` (Pi 4, Pi 400 y CM4) y los `bcm2712-*` (Pi 5, Pi 500 y CM5). Junto a ellos aparecen dos imágenes de kernel, `kernel8.img` y `kernel_2712.img`, y el firmware propietario del GPU (`bootcode.bin`, los archivos `start*.elf` y sus correspondientes `fixup*.dat`).

Esto explica el mecanismo real de arranque: una sola imagen de tarjeta sirve para varios modelos porque el firmware, al iniciar, identifica la revisión de la placa y selecciona el `.dtb` y la imagen de kernel correctos. También se observó el directorio `overlays/`, que contiene los fragmentos de árbol de dispositivos que `config.txt` puede activar para habilitar periféricos opcionales, y los archivos `meta-data`, `network-config` y `user-data`, propios del mecanismo de aprovisionamiento *cloud-init*.

El listado de `/proc/device-tree/` (Fig. 4) confirmó que el kernel interpretó ese descriptor binario y expuso cada nodo como un directorio navegable. Entre los nodos se identificaron `model` y `compatible`, que identifican la placa; `cpus` y `memory@0`, que declaran los recursos de cómputo y memoria; `soc`, que agrupa los periféricos integrados; y nodos de hardware específico como `gpu`, `v3dbus`, `leds`, `thermal-zones`, `timer`, los reguladores fijos `fixedregulator_3v3` y `fixedregulator_5v0`, y los relojes y reguladores de las interfaces de cámara (`cam0_clk`, `cam1_clk`, `regulator-cam1`).

Esto ilustra de forma concreta el concepto de BSP visto en la Unidad 2: en ARM el kernel no descubre el hardware por enumeración automática, como ocurre en PCI o USB, sino que recibe su descripción explícita en el momento del arranque. Los nodos `__symbols__` y `__overrides__` son precisamente los puntos de anclaje que permiten a los overlays modificar ese árbol sin recompilarlo por completo.

IV-C. Procesos y servicios

La salida de `ps aux` (Fig. 5) mostró en la primera posición el proceso con PID 1, `/sbin/init splash`, que en este sistema es `systemd` y actúa como ancestro de todos los procesos de usuario. Inmediatamente después aparece una larga secuencia de procesos cuyos nombres están entre corchetes (`[kthreadd]`, `[kworker/...]`, `[ksoftirqd]`, `[migration]`, `[rcu_...]`) y cuyas columnas `VSZ` y `RSS` valen cero. Son hilos del kernel: se planifican como cualquier otro proceso, pero no tienen espacio de direcciones de usuario, por lo que no ocupan memoria virtual propia.

Los estados observados en la columna `STAT` fueron principalmente `S` (*sleeping*, equivalente al estado bloqueado del diagrama visto en clase), `I` e `I<` (hilos de kernel ociosos, el segundo con prioridad elevada por *nice* negativo), `R` (en ejecución) y `Ss` (en espera, además líder de sesión).

El monitor `top` (Fig. 6) cuantificó esa observación de manera contundente: de 185 tareas totales, 184 estaban durmiendo y solamente 1 en ejecución, sin procesos detenidos ni zombis, con el CPU 99.7 % inactivo y un *load average* de 0.00. El único proceso en estado `R` era el propio `top`. Este es el comportamiento normal de un sistema multitarea: la inmensa mayoría de los procesos permanece bloqueada esperando un evento, y el planificador solo necesita repartir el CPU entre los pocos que están listos.

En cuanto a memoria, `top` reportó 905.7 MiB totales, con 181.4 MiB libres, 332.8 MiB en uso y 471.3 MiB en *buffers* y caché, además de 905.0 MiB de espacio de intercambio completamente sin utilizar. Vale la pena notar que la memoria dedicada a caché no está desperdiciada: el kernel la libera automáticamente cuando una aplicación la necesita, razón por la cual la métrica *available* (573.0 MiB) es más significativa que *free*.

El comando `systemctl status` (Fig. 7) reportó el sistema en estado *running*, con 405 unidades cargadas, cero trabajos en cola y cero unidades fallidas, bajo `systemd` 257.13. En el árbol de grupos de control se distinguen `init.scope`, que contiene al PID 1, y `system.slice`, que agrupa los servicios del sistema: `NetworkManager`, `ssh` (con `sshd` escuchando conexiones, que es justamente el servicio que hace posible esta sesión de trabajo), `avahi-daemon` anunciando el nombre `konan.local` por mDNS, `bluetooth`, `cron`, `dbus`, `lightdm`, `polkit`, `systemd-journald`, `systemd-logind` y `systemd-timesyncd`.

Un dato llamativo fue la marca temporal de arranque: *Since: Wed 1969-12-31 18:00:02 CST*. La Raspberry Pi carece de reloj de tiempo real con batería, por lo que al encender parte del inicio de la época Unix (1 de enero de 1970 UTC, que en la zona horaria local CST corresponde al 31 de diciembre de 1969 a las 18:00) hasta que `systemd-timesyncd` sincroniza la hora por NTP. En un sistema embebido sin conexión de red permanente, esta limitación obliga a incluir un módulo RTC externo si la aplicación depende de marcas de tiempo confiables.

Finalmente, el listado de unidades de servicio (Fig. 8) evidenció la diferencia entre los

subestados `running` y `exited`. Servicios como `cron.service` o `dbus.service` permanecen en `running` porque mantienen un demonio vivo; en cambio `alsa-restore.service`, `console-setup.service`, `keyboard-setup.service` y las cuatro etapas de `cloud-init` aparecen como `active (exited)`: son servicios de tipo *oneshot* que ejecutaron su tarea de configuración y terminaron, pero `systemd` los mantiene marcados como activos para registrar que ya se cumplieron.

IV-D. Programa en C

El programa `reporte_sistema` compiló sin advertencias ni errores y produjo los valores de la Tabla II (Fig. 10).

Cuadro II
DATOS OBTENIDOS POR `REPORTE_SISTEMA`.

Campo	Valor obtenido
Arquitectura	`aarch64`
Kernel	`6.18.39+rpt-rpi-v8`
Model	Raspberry Pi 4 Model B Rev 1.5
MemTotal	927 444 kB
MemFree	180 044 kB
Memoria libre	175.8 MB

Los valores coincidieron con los obtenidos previamente mediante los comandos del sistema. El total de 927 444 kB equivale a 905.7 MiB, exactamente la cifra que reportó `top` como memoria total, y la arquitectura `aarch64` es la misma que declaró `lscpu`. Esta correspondencia confirma que el programa no está estimando nada: lee la misma fuente que consultan las utilidades del sistema.

Conviene subrayar que `/proc` no es un sistema de archivos en disco. Sus archivos se generan en memoria por el kernel en el instante de la lectura, razón por la cual `MemFree` varía entre ejecuciones consecutivas del programa (180 044 kB al ejecutar el programa contra los 181.4 MiB que había reportado `top` minutos antes), mientras que `MemTotal` permanece constante.

Un resultado inesperado fue que la etiqueta `Hardware` no produjo ninguna salida. La causa está en el diseño de `mostrar_linea()`: la función no reposiciona el descriptor al inicio del archivo, de modo que la segunda llamada continúa la lectura desde donde terminó la primera. Como la línea `Model` se encuentra al final de `/proc/cpuinfo`, la búsqueda posterior de `Hardware` alcanza el final del archivo sin encontrar coincidencia. La solución es invocar `rewind()` antes de cada búsqueda, o bien reordenar las llamadas según el orden real del archivo. Este comportamiento es un buen recordatorio de que un descriptor de archivo mantiene estado entre llamadas.

Respecto al reto opcional, la conversión de kilobytes a megabytes se calculó como:

$$\text{MemFree}_{\text{MB}} = \frac{180\,044 \text{ kB}}{1024} = 175.82 \text{ MB} \qquad (1)$$

El uso del divisor `1024.0` en lugar de `1024` es indispensable: con dos operandos enteros, C realizaría división entera y truncaría el resultado a 175, perdiendo la parte fraccionaria. Al escribir el divisor como literal de punto flotante se fuerza la promoción del operando entero a `double` y se conserva la precisión.

IV-E. GPIO

El listado de `/sys/class/gpio/` (Fig. 11) devolvió seis entradas: `chip0`, `chip1`, `export`, `gpiochip512`, `gpiochip570` y `unexport`. Los archivos `export` y `unexport` corresponden a la antigua interfaz *sysfs*, en la que se escribía el número de una línea para reservarla desde el espacio de usuario; esta interfaz está formalmente obsoleta y ha sido sustituida por los dispositivos de caracteres `/dev/gpiochip*` y la biblioteca `libgpiod`.

Las dos entradas numeradas representan los dos controladores presentes en la Pi 4: el del propio SoC BCM2711, con sus 58 líneas a partir de la base 512, y el expansor de GPIO gestionado por el firmware, que arranca justo después en 570. Es importante notar que estas bases son asignadas dinámicamente por el kernel y han cambiado entre versiones, por lo que codificar los números absolutos en un programa es frágil. La forma robusta de referirse a una línea es por su controlador y desplazamiento, que es precisamente lo que ofrece `libgpiod` y lo que se utilizará al cablear hardware en la Práctica 2.

IV-F. Script de diagnóstico

El script generó correctamente el archivo `sysinfo_20260907_204455.log` (Figs. 13 y 14), cuyo nombre incorpora la marca temporal producida por `date +%Y%m%d_%H%M%S`. Esta convención evita que ejecuciones sucesivas se sobrescriban entre sí, algo deseable cuando se quiere comparar el estado del sistema en distintos momentos.

La información recopilada complementó la de las secciones anteriores. De `lscpu` se obtuvo el detalle del procesador: cuatro núcleos ARM Cortex-A72 en línea (0–3), un solo hilo por núcleo, ordenamiento *little endian* y soporte para modos de operación de 32 y 64 bits, lo que significa que el SoC puede ejecutar binarios ARMv7 además de los ARMv8 nativos. De `df -h /` se confirmó que la raíz reside en `/dev/mmcblk0p2`, la segunda partición de la tarjeta microSD, con 59 GB de capacidad, 6.0 GB usados y 11 % de ocupación. La primera partición es la de arranque explorada en la Parte B, montada en `/boot/firmware`.

En la sección de procesos ordenados por consumo de CPU apareció una anomalía instructiva: el propio comando `ps aux --sort=-%cpu` encabezó la lista con 300 % de CPU. Esto no indica una sobrecarga real, sino un artefacto de medición: `ps` calcula el porcentaje como el cociente entre el tiempo de CPU consumido y el tiempo de vida del proceso, y como el proceso acababa de nacer, ese denominador es minúsculo. Los demás procesos de la lista (`wf-panel-pi`, `pcmanfm`, `labwc` y el propio `init`) pertenecen al entorno gráfico de escritorio basado en Wayland, y son los responsables de la mayor parte de los 310 MiB de memoria en uso.

IV-G. Control de versiones

El comando `git log` (Fig. 15) confirmó la creación del commit `c891cf9b0baacf3e94b7527786beb91882da14aa` sobre la rama `master`, con fecha del 7 de septiembre de 2026 y el mensaje `Practica 1` acompañado de los nombres de los seis integrantes del equipo. El resumen del commit reportó dos archivos modificados y 74 líneas insertadas.

Un detalle técnico interesante son los modos de archivo registrados: `100644` para `reporte_sistema.c` y `100755` para `sysinfo.sh`. Git no almacena los permisos completos de UNIX, pero sí conserva el bit de ejecución, de modo que el `chmod +x` aplicado en la Parte F queda versionado junto con el contenido. Al clonar el repositorio en otra máquina, el script vendrá ya ejecutable y no habrá que repetir el comando.

También cabe señalar que el binario `reporte_sistema` y los archivos `.log` quedaron deliberadamente fuera del control de versiones: son artefactos regenerables a partir del código fuente y su inclusión solo inflaría el historial. En prácticas posteriores convendrá formalizar esta exclusión mediante un archivo `.gitignore`.

IV-H. Problemas encontrados

Cuadro III
INCIDENCIAS DURANTE LA PRÁCTICA Y SU SOLUCIÓN.

Problema	Solución aplicada
El commit inicial quedó registrado con una identidad de autor incorrecta.	Configurar `user.email` y rehacer el commit con `git commit --amend --reset-author`.
La etiqueta `Hardware` no apareció en la salida del programa en C.	El descriptor de archivo quedaba posicionado después de `Model`; se resuelve con `rewind()` antes de cada búsqueda.
El reloj del sistema marcaba el 31/12/1969 como fecha de arranque.	La placa no tiene RTC con batería; la hora se corrige al sincronizar por NTP con `systemd-timesyncd`.

V. CONCLUSIONES

Belmont Muñoz Samuel. El desarrollo de esta práctica permitió comprender de manera clara y directa cómo opera Linux en un sistema embebido al interactuar con los recursos propios de la tarjeta. Resultó especialmente formativo comprobar cómo el sistema operativo organiza y muestra la información del hardware a través de sus archivos internos, así como verificar que un programa en C o un script de Bash pueden consultar directamente esos datos de forma confiable. Además, resolver los detalles prácticos durante la compilación y la lectura de archivos ayudó a valorar la atención al detalle que exige la programación cercana al sistema. Cabe mencionar que la integración de scripts para automatizar tareas y el uso de Git desde esta primera etapa sientan una base de trabajo ordenada y funcional para el resto de las prácticas.

Domínguez Palacios Jesús Alejandro. Lo más valioso de esta práctica fue comprobar que un sistema Linux embebido no oculta su hardware, sino que lo publica de forma legible a través de `/proc`, `/sys` y `/proc/device-tree`. Escribir `reporte_sistema.c` dejó claro que utilidades como `free` o `lscpu` no hacen magia: leen exactamente los mismos archivos virtuales que yo abrí con `fopen()`, y por eso mis 927 444 kB coincidieron con los 905.7 MiB que reportó `top`. El detalle de la etiqueta `Hardware` que nunca se imprimió fue el que más me enseñó, porque me obligó a razonar sobre el estado interno del descriptor de archivo en vez de suponer que cada llamada empieza desde cero. También me quedó clara la diferencia frente al trabajo con microcontroladores sin sistema operativo, donde no existe esta capa de introspección y el acceso al hardware se hace escribiendo directamente en registros. Finalmente, iniciar el repositorio de Git desde la primera práctica establece un hábito de trabajo que evitará tener que reconstruir el historial más adelante.

Dorantes Mejía Emmanuel. Durante esta práctica pude familiarizarme mejor con el entorno de trabajo de la Raspberry Pi y con algunas herramientas fundamentales para el desarrollo en Linux. La instalación y actualización de paquetes, así como el uso de herramientas como GCC, Make y Git, permitió preparar correctamente el sistema para trabajar en las siguientes actividades. El desarrollo del programa en C fue útil para conocer cómo se puede obtener información directamente del sistema, como datos del procesador, memoria y arquitectura, utilizando los recursos que proporciona Linux. De igual manera, la creación de un script en Bash mostró cómo es posible automatizar diferentes comandos y facilitar tareas de monitoreo. Finalmente, el uso de Git y la creación del repositorio resultan importantes para mantener un mejor control de los archivos y cambios que se realicen durante las próximas prácticas.

García Olvera Emiliano. Esta práctica me ayudó a entender de forma concreta cómo funciona por dentro la Raspberry Pi: qué tan actualizado está su sistema, qué procesos corren en todo momento y cómo el sistema operativo detecta y organiza el hardware conectado. Programar en C para leer datos reales del equipo, en vez de solo verlos en pantalla, hizo que los conceptos se sintieran mucho más claros. También me quedó claro el valor de automatizar tareas con scripts y de usar Git desde ahora, ya que este mismo repositorio lo seguiremos usando durante el semestre.

Hernández Alonso Antonio Adonahí. En esta práctica di los primeros pasos dentro del entorno de Linux embebido en la Raspberry Pi como lo fue la actualización el sistema y gestión de paquetes con apt, importante para cualquier tipo de instalación como en los toolchains de desarrollo que ocuparemos. Desarrollar el programa reporte_sistema en C me hizo interactuar directamente con las fuentes reales de información del sistema como la arquitectura, CPU y memoria, mientras que el script en bash me recordó la utilidad de combinar comandos de diagnóstico en una herramienta reutilizable. Todo esto ahora conectado con git que nos permitirá trabajar de manera colaborativa y remota para el resto de las prácticas que se tendrán más adelante.

Jiménez Reyes Alan Yael. Durante esta práctica, logramos consolidar nuestro entorno de desarrollo directamente en la

Raspberry Pi. Me pareció fundamental el proceso de instalar el toolchain de compilación (`gcc`, `make`) y Git, ya que nos prepara para trabajar de forma nativa en este sistema de Linux embebido. Lo que más me llamó la atención fue el desarrollo del programa en C (`reporte_sistema.c`), pues nos permitió entender cómo el sistema operativo expone la información real del hardware interactuando directamente con el sistema de archivos virtual en `/proc`. Además, la creación del script en Bash (`sysinfo.sh`) me ayudó a comprender cómo podemos automatizar el monitoreo agrupando comandos de diagnóstico. Finalmente, inicializar nuestro repositorio y hacer nuestro primer commit nos deja una base sólida y bien estructurada para gestionar el código en el resto de las prácticas del semestre.

Durante la elaboración de este reporte se utilizó el asistente de inteligencia artificial Claude [7] como apoyo para la redacción; todos los procedimientos, capturas y resultados presentados fueron ejecutados y verificados por los integrantes del equipo.

REFERENCIAS

[1] Free Software Foundation. (2024). "GCC, the GNU Compiler Collection", *GNU Project*. Disponible en: https://gcc.gnu.org/

[2] The kernel development community. (2024). "The /proc Filesystem", *Linux Kernel Documentation*. Disponible en: https://www.kernel.org/doc/html/latest/filesystems/proc.html

[3] Free Software Foundation. (2024). "Bash Reference Manual", *GNU Project*. Disponible en: https://www.gnu.org/software/bash/manual/

[4] Raspberry Pi Ltd. (2024). "Raspberry Pi Documentation — The config.txt file", Disponible en: https://www.raspberrypi.com/documentation/computers/config-txt.html

[5] Linux GPIO subsystem. (2024). "GPIO Character Device Userspace API", *Linux Kernel Documentation*. Disponible en: https://www.kernel.org/doc/html/latest/userspace-api/gpio/

[6] S. Chacon y B. Straub. (2014). *Pro Git*, 2a ed. Apress. Disponible en: https://git-scm.com/book/

[7] Anthropic. (2026). "Claude". Disponible en: https://claude.ai/